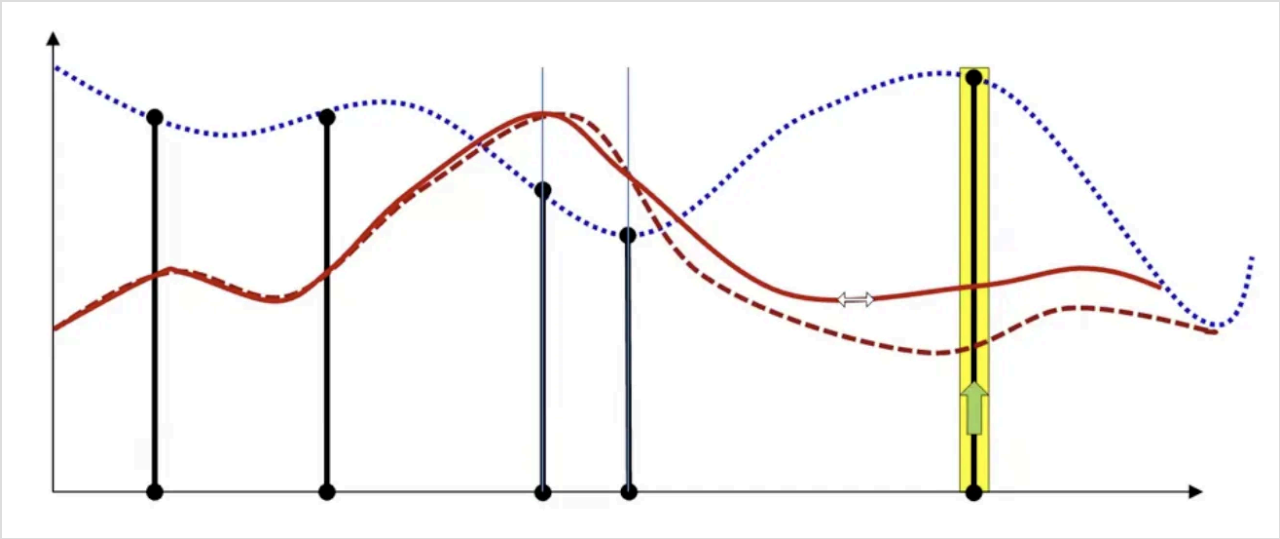
Lec7. Training Part V

Training Neural Networks : Optimization

Alternative : Incremental update



- 각 점마다, 따로 **training** 해감.
- → 국소적인 수정을 계속 하면서, error를 줄여나감

Given $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \ldots, W_K$

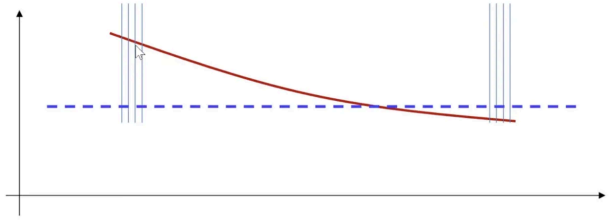
Do:

- For all $t = 1:T$
  - For every layer k:
    - Compute $\nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})$
    - Update

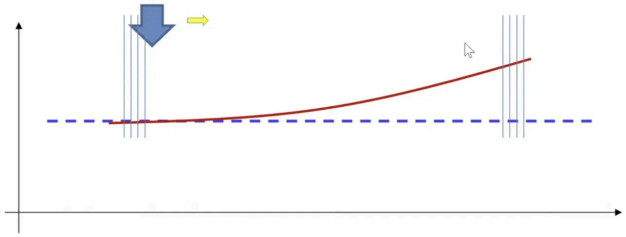
$$W_k = W_k - \eta \nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})^T$$

Until $Loss$ has converged

Caveats: order of presentation



Caveats: order of presentation



- 이 경우, 위와 같이 iteration할 수 있음
- ⇒ 따라서, **random order**로 어떤 데이터를 학습할지 정해야 함.

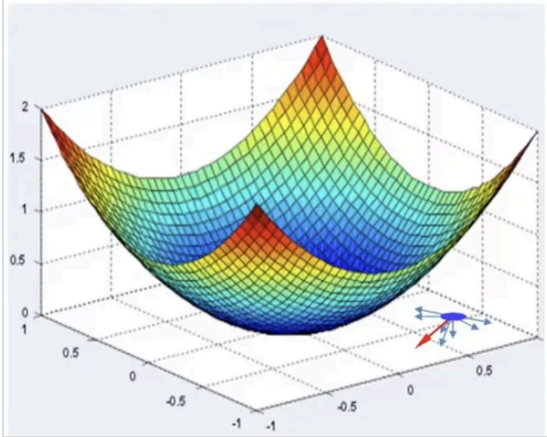
Do:

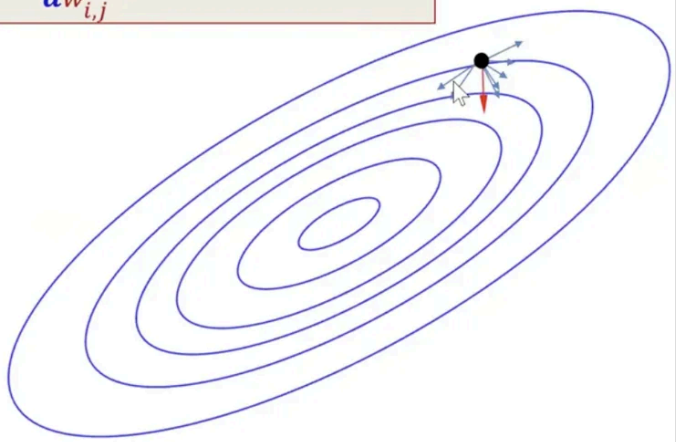
- Randomly permute $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$
- For all $t = 1:T$
  - For every layer k:
    - Compute $\nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})$
    - Update

$$W_k = W_k - \eta \nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})^T$$

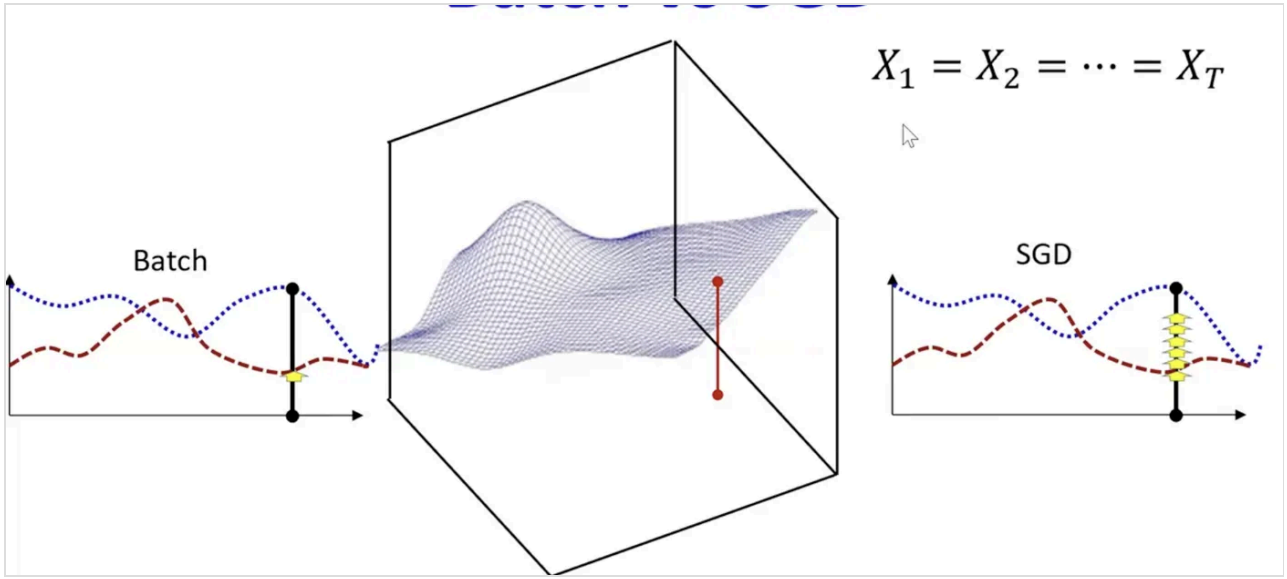
The expected behavior of the gradient

$$\frac{d\boldsymbol{E}(\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \ldots, \boldsymbol{W}^{(K)})}{d\boldsymbol{w}_{i,j}^{(k)}} = \frac{1}{\boldsymbol{T}} \sum_{\boldsymbol{i}} \frac{d\boldsymbol{Div}(\boldsymbol{Y}(\boldsymbol{X_i}), \boldsymbol{d_i}; \boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \ldots, \boldsymbol{W}^{(K)})}{d\boldsymbol{w}_{i,j}^{(k)}}$$

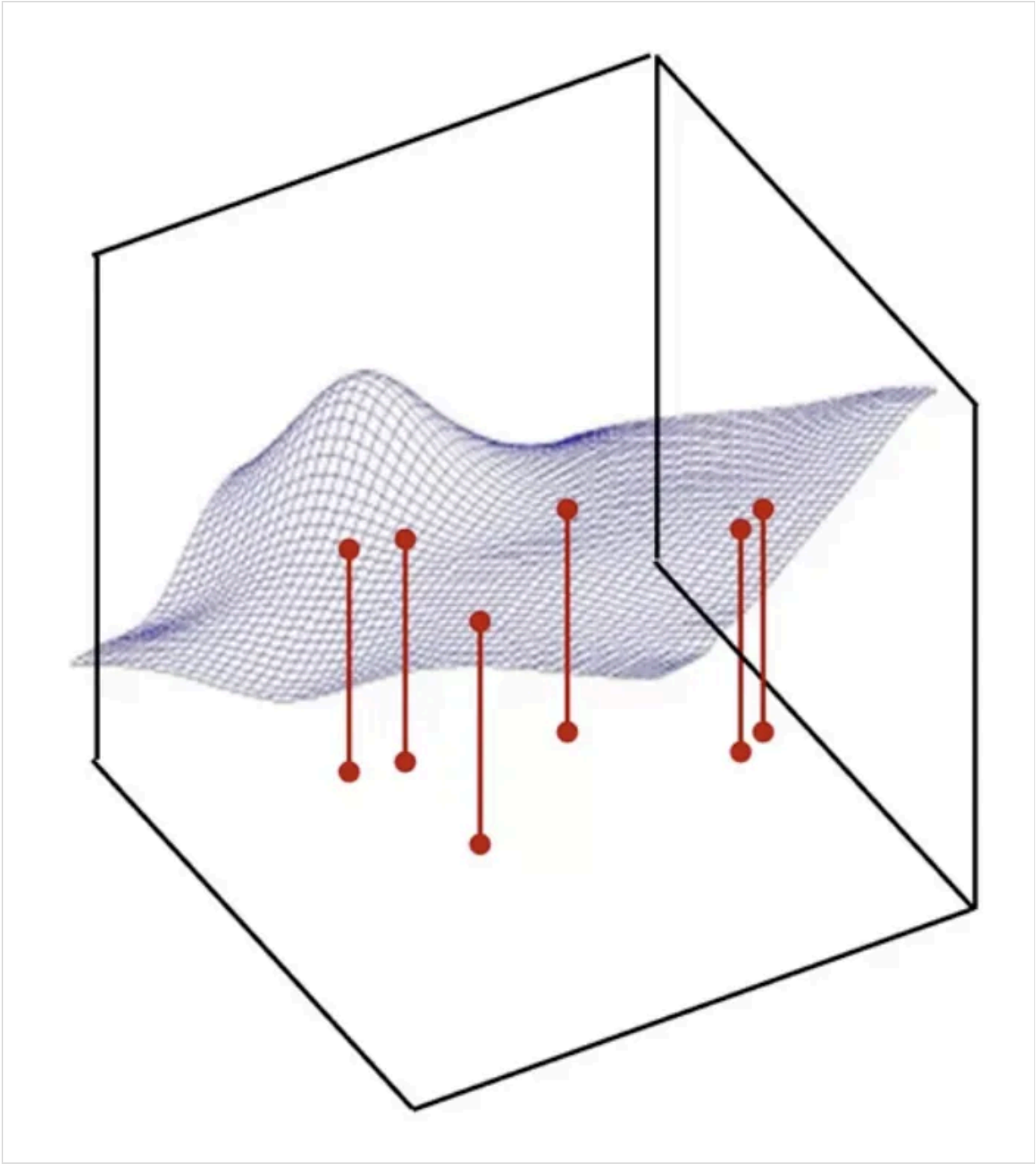




- 각 w의 편미분을 학습하게 됨
  - ⇒ 모두 다른 방향으로 학습하게 됨
    ⇒ 평균 미분값으로 학습됨.



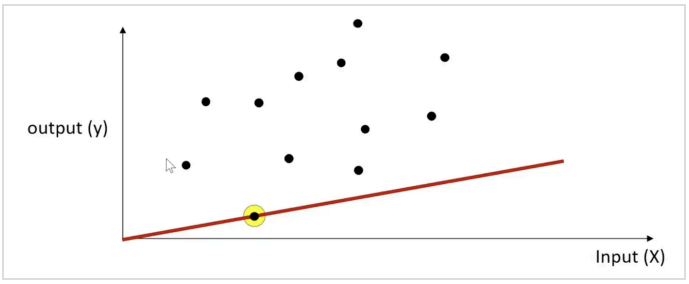
- 모든 데이터가 같은 곳에 뭉쳐있다면?
  - → 모든 데이터의 평균값을 계산한다면,
    ⇒ 하나의 그래디언트를 구하는 것과 같게 됨.
- Batch : 전체 평균(계산은 전체, 평균값 1번 반환)만큼 GD를(한 번) 하게 됨.
  SGD : 한 점을 여러번(전체만큼) GD하게 됨.
  - → SGD는 Batch와 같은 연산량으로 여러번 업데이트를 하게 됨.
- 데이터가 divergence할수록 업데이트의 효과는 낮아지게 된다.

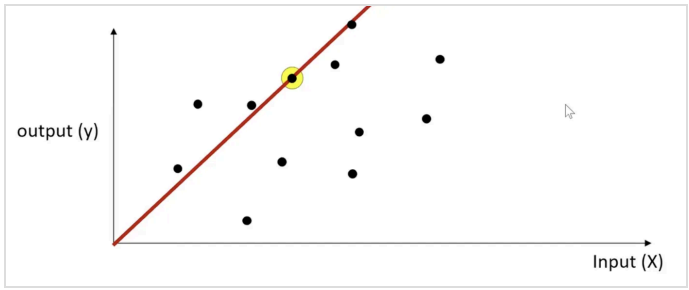


- **data**가 **divergence**해진다면?
  - → Batch에서 평균값은 0으로 (거의)수렴하게 됨.
- 따라서, Batch보다 SGD가 효율적임.

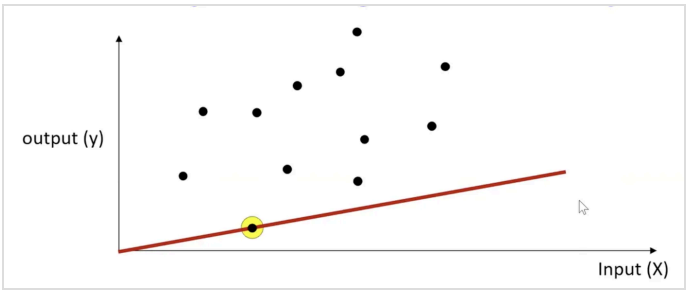
Incremental learning runs the risk of always chasing the latest input

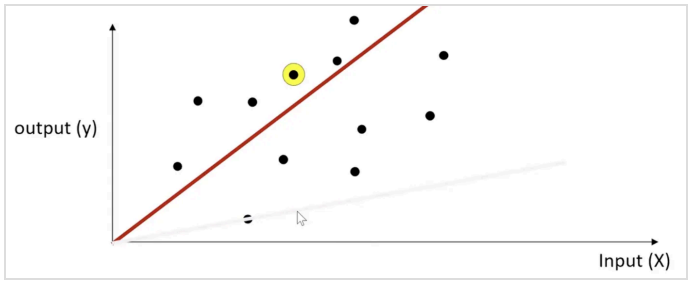
---



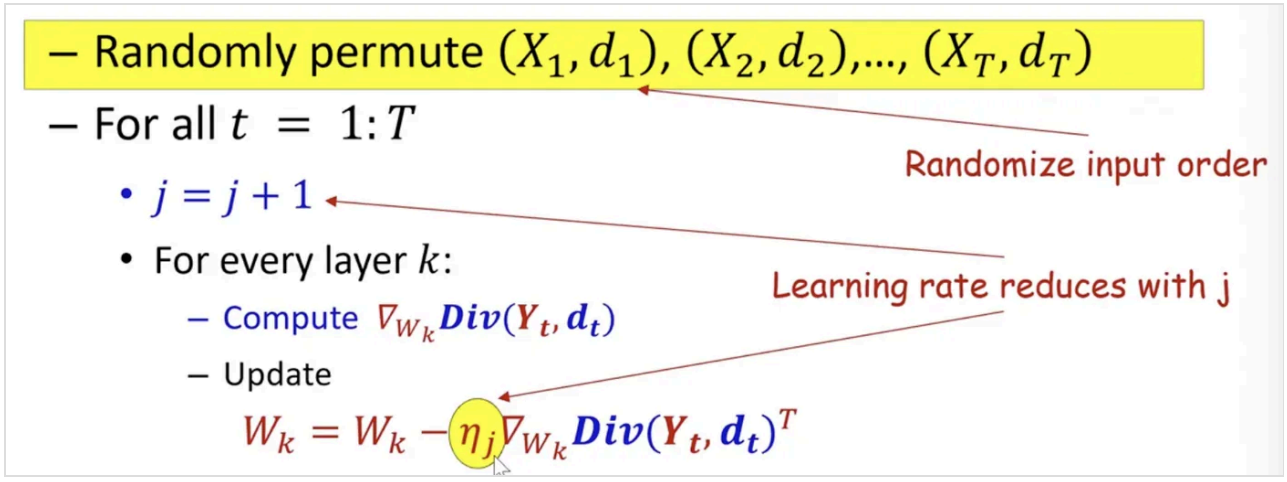


- SGD를 한다면,
  data를 하나 → 하나 → … 이렇게 뽑아서, 오차를 최소화 함.
  = 가장 최신의 오차만 학습하게 됨.
  → 즉 가장 최근 데이터에 **과적합** 될 수 있음.





Learning rate를 줄여가면서 학습하면 됨.→ 그러면 어느 선으로 수렴하게 됨.



SGD convergence (SGD 수렴 조건)

- Sufficient condition: step sizes follow the following conditions (Robbins and Munro 1951)

$$\sum_k \eta_k = \infty$$

  - Eventually the entire parameter space can be searched

$$\sum_k \eta_k^2 < \infty$$

  - The steps shrink
- The fastest converging series that satisfies both above requirements is

$$\eta_k \propto \frac{1}{k}$$

  - This is the optimal rate of shrinking the step size for strongly convex functions
- More generally, the learning rates are heuristically determined

- SGD는 확률적인 Gradient를 사용하므로 노이즈가 발생한다. 따라서 **학습률(Learning Rate)을 적절히 감소시켜야 수렴을 보장**할 수 있다.

1. Robbins-Monro Conditions

> SGD 수렴을 위한 학습률 조건:

- $\sum_{k=1}^{\infty} \eta_k = \infty$

> 학습률이 너무 빨리 감소하면 안 됨.
> 최적해까지 충분히 이동할 수 있어야 함.
> $\sum_{k=1}^{\infty} \eta_k^2 < \infty$

- 학습률을 충분히 빠르게 감소시켜야 함.
  확률적 Gradient에서 발생하는 노이즈의 누적 영향을 제한.
- 2. Learning Rate Schedule
  $\eta_k = \frac{1}{k^p}$

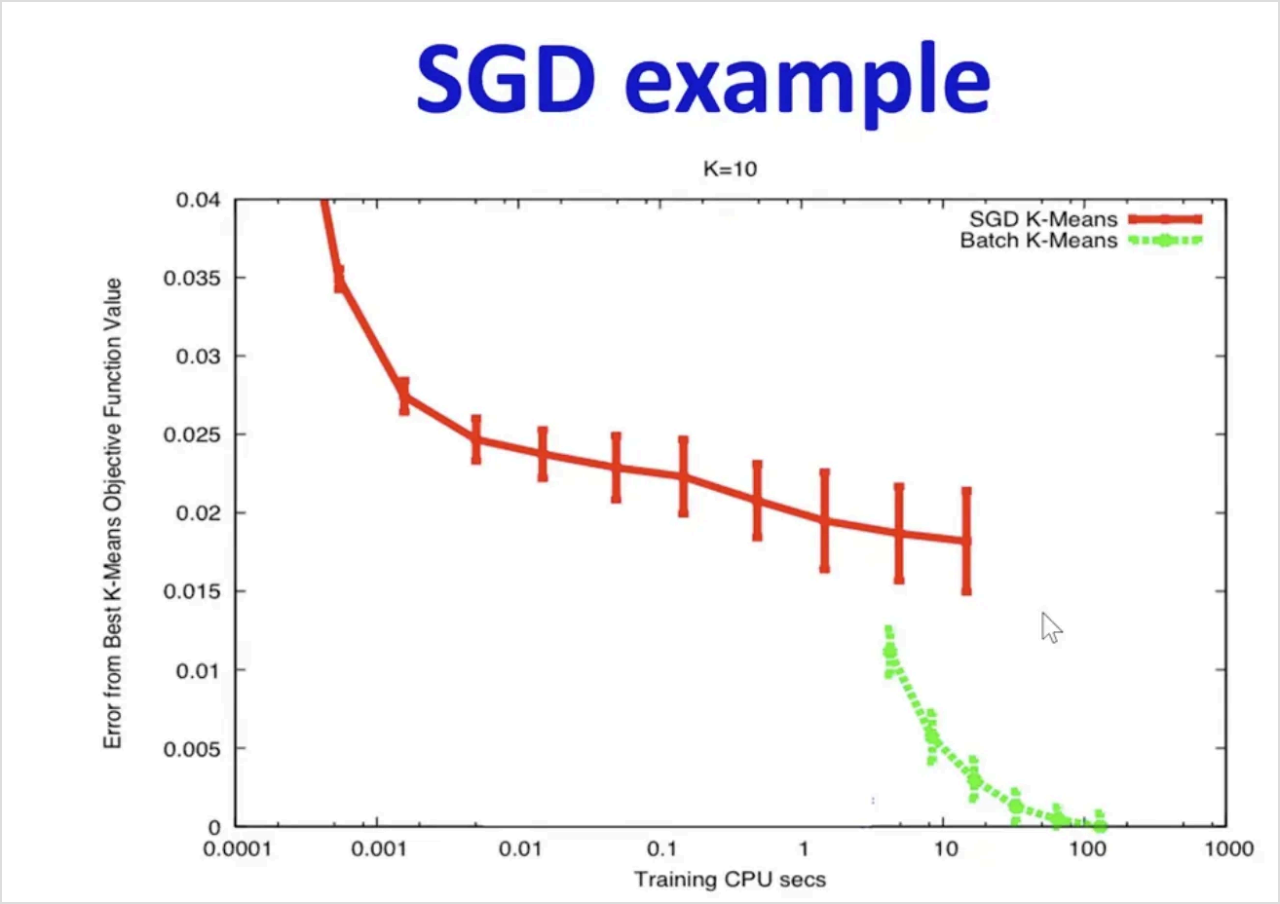
> 두 조건을 동시에 만족하려면:
> $\frac{1}{2} < p \leq 1$

- 대표적으로 다음을 사용할 수 있음.
  $\eta_k \propto \frac{1}{k}$
- 3. Convex vs. Non-convex

> Convex Loss: 적절한 조건에서 전역 최적해(Global Minimum)로 수렴.

- Non-convex Loss: 전역 최적해는 보장되지 않으며, 일반적으로 정상점(Stationary Point)으로의 수렴을 분석함.

- SGD example



- SGD : 빠름, but 분산이 크고 / 잡음이 많음.
  Batch : 느림, 조금씩 움직이며 거의 0으로 수렴함

- Modelling a function

$$\widehat{W} = \underset{W}{\operatorname{argmin}} \int_X div\big(f(X;W), g(X)\big) P(X) dX$$
$$= \underset{W}{\operatorname{argmin}}\, E\big[div\big(f(X;W), g(X)\big)\big]$$

- 최종적으로 minimize하고 싶은 것은 **integral of div / expected div**

$$Loss(W) = \frac{1}{N} \sum_{i=1}^{N} div(f(X_i;W), d_i)$$
$$\widehat{W} = \underset{W}{\operatorname{argmin}}\, Loss(W)$$

> Empirical Risk Minimization (ERM)

- 최종 목표 : **Expected Risk (전체 데이터 분포의 평균 오차) 최소화**

Expected Risk

- R(W) = E $$div(f(X;W), d)$$
  - → 실제 데이터 분포 전체에 대한 평균 Loss

Empirical Risk

- Loss(W) = (1/N) Σ_i div(f(X_i; W), d_i)
  - → 실제 분포를 알 수 없으므로, **학습 데이터의 평균 Loss로 근사**
  - → $\hat{W}$ = argmin Loss(W)
  - 즉, 평균 Loss를 최소화하는 W를 찾는 것.

SGD : Unbiased Estimate

- SGD : 매 iteration마다 **하나의 데이터만 선택하여 업데이트**
  - $L_i(W) = div(f(X_i; W), d_i)$

Sample Loss의 기댓값 = 전체 평균 Loss

- E

$$L_i\ (W)$$

  = $(1/N)\ \Sigma_i\ L_i(W) = Loss(W)$

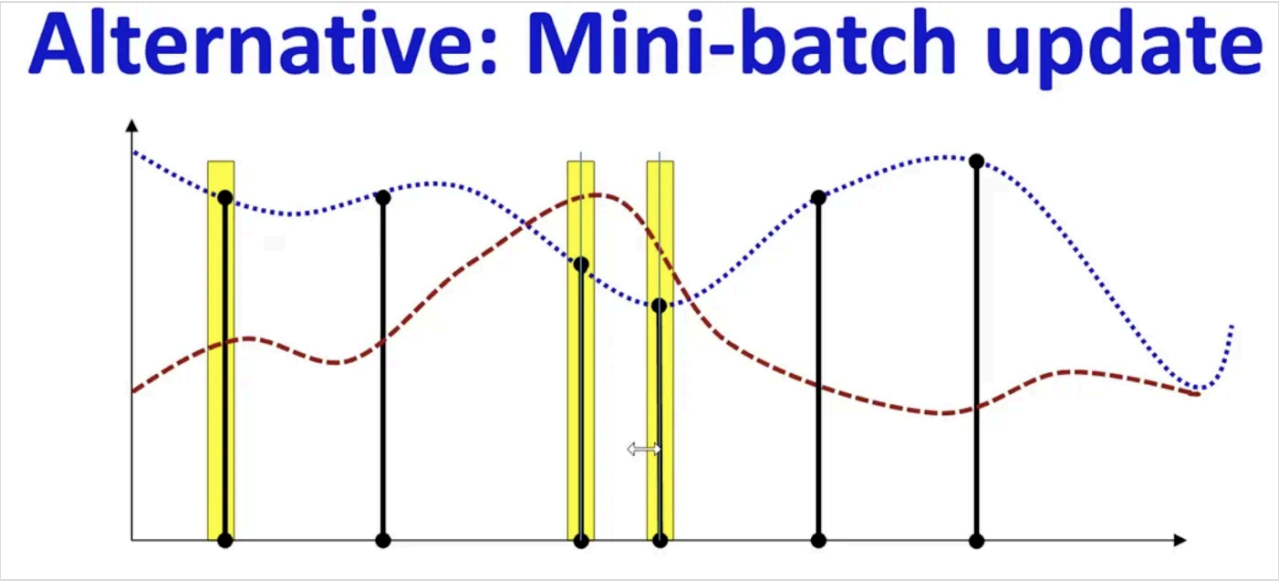
Gradient도 마찬가지

- E

$$\nabla L_i\ (W)$$

  = $\nabla Loss(W)$
  - → 이를 **Unbiased Estimate** 라고 함.
  - ⇒ 개별 Gradient에는 노이즈가 있지만, 평균적으로 전체 Gradient와 같음.
- ⇒ 따라서 **전체 데이터를 매번 계산하지 않고도 SGD로 학습 가능.**

SGD의 단점 해결 → Minibatch



- 무작위 n개를 선택해서, 학습에 이용함.

Given $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \ldots, W_K$; $j = 0$

Do:

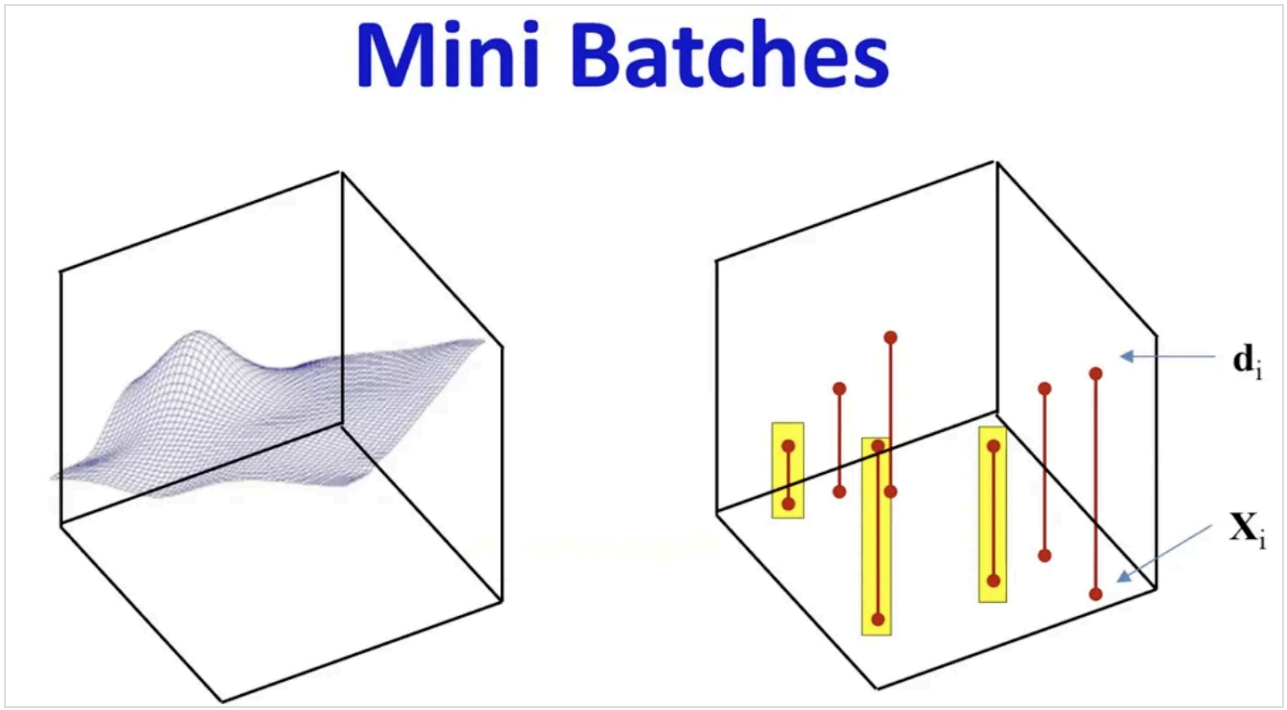
- Randomly permute $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$
- For $t = 1 : b : T$
  - $j = j + 1$
  - For every layer k:
    - $\Delta W_k = 0$
  - For t' = t : t+b-1
    - For every layer k:
      - » Compute $\nabla_{W_k} Div(Y_t, d_t)$
      - » $\Delta W_k = \Delta W_k + \frac{1}{b} \nabla_{W_k} Div(Y_t, d_t)^T$
  - Update
    - For every layer k:

$$W_k = W_k - \eta_j \Delta W_k$$

Until Err has converged

Mini-batch size

Shrinking step size

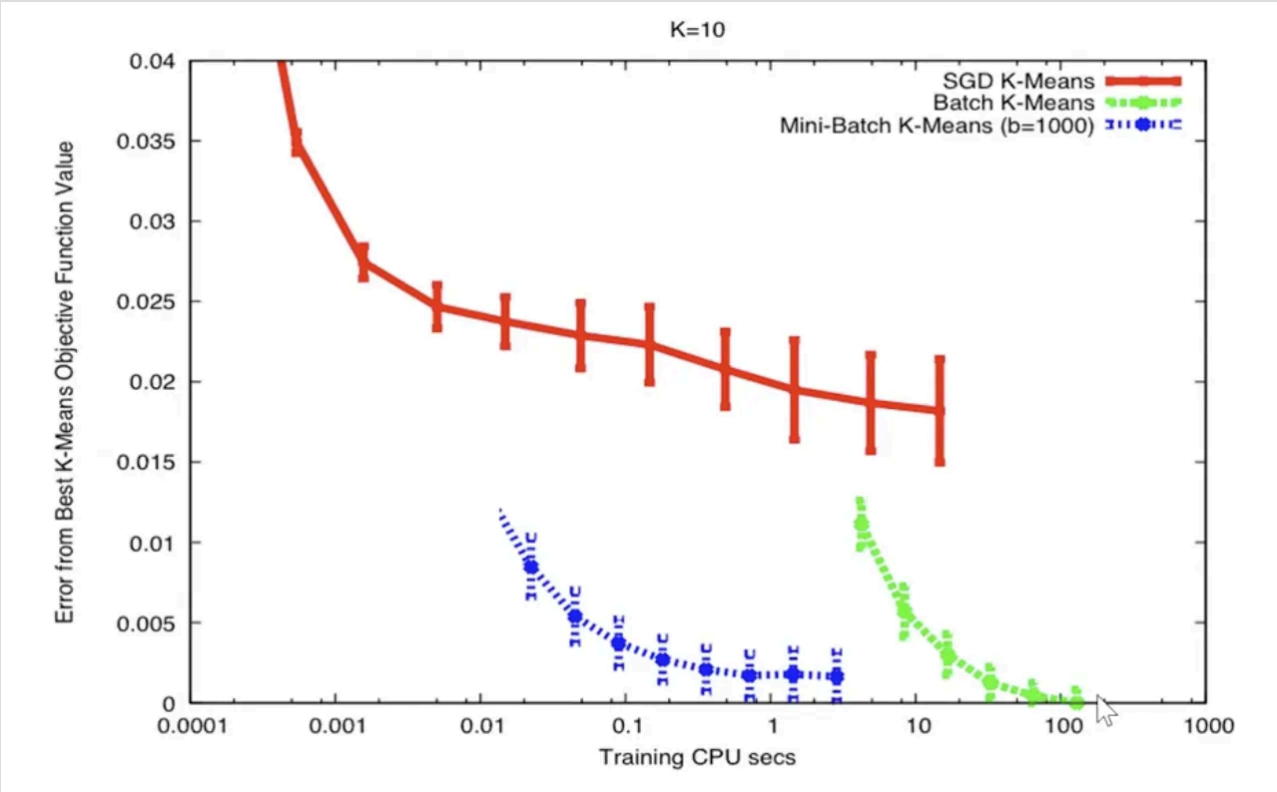


Mini-batch Loss

- $MiniBatchLoss(W) = \frac{1}{b} \sum_{i=1}^{b} div(f(X_i; W), d_i)$

Expected Mini-batch Loss

- $\mathbb{E}[MiniBatchLoss(W)] = \mathbb{E}[div(f(X;W), g(X))]$
- minibatch 사이즈에 따라서,
  SGD / Batch 사이의 어떤 점의 특성을 가지게 됨.
  b(minibatch size)가 커지면 → Var(div)가 줄어듦



Trend algorithms

RECALL : Momentum Update

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)}\right)^T$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

- Typical β value is 0.9
  - maintains a running average

RECALL : Nesterov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)} + \beta \Delta W^{(k-1)}\right)^T$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

RMSProp (Root Mean Square Propagation)

- 각 파라미터마다 **Learning Rate를 다르게 조절**하는 최적화 알고리즘.

1. 아이디어

- Gradient가 큰 파라미터 → Learning Rate 감소
  Gradient가 작은 파라미터 → Learning Rate 상대적으로 증가
  Gradient 제곱의 이동평균을 이용함.

2. 수식

- Gradient 제곱의 이동평균:
  - $v_t = \beta v_{t-1} + (1-\beta) g_t^2$
- 파라미터 업데이트:
  - $w_{t+1} = w_t - \eta g_t / (\sqrt{v_t} + \varepsilon)$
- ※ g_t^2는 Gradient의 제곱이지, 2차 미분이 아님.
- **⇒ 파라미터별 업데이트 크기를 조절하여 SGD보다 안정적인 학습을 유도함.**

- **Adam (Adaptive Moment Estimation)**
  Momentum + RMSProp 결합

1. Momentum (Gradient 이동평균)

- $m_k = \delta m_{k-1} + (1-\delta) g_k$
  → 업데이트 방향 안정화

2. RMSProp (Gradient 제곱 이동평균)

- $v_k = \gamma v_{k-1} + (1-\gamma) g_k^2$
  - → 파라미터별 Learning Rate 조절

3. Bias Correction

- $\hat{m}_k = m_k / (1-\delta^k)$
  $\hat{v}_k = v_k / (1-\gamma^k)$
  - → 초기 이동평균의 편향 보정

4. Parameter Update

- $w_{k+1} = w_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \varepsilon)$
  - **⇒ 방향 안정화 + 학습률 자동 조절로 효율적인 최적화 수행.**